LAB REPORT

Fundamentals of Operating Systems (OS) Assignment – 3

Name : Dhruv Gupta

Roll No : 2401201015

Course : BCA (AI & DS) [H]

Section : A

---

1. Introduction

In modern operating systems, virtual memory plays a crucial role in efficient memory management. Since the physical memory (RAM) is limited, systems use paging to manage memory effectively. When all memory frames are occupied and a new page needs to be loaded, a **page replacement algorithm** is used to decide which page should be removed.

This assignment focuses on implementing and analyzing multiple page replacement algorithms, including:

- FIFO (First-In-First-Out)
- LRU (Least Recently Used)
- Optimal Page Replacement
- MRU (Most Recently Used)
- Second Chance (Clock Algorithm)

The performance of each algorithm is evaluated based on the number of page faults generated.

2. Objectives

- To understand the concept of virtual memory and paging
- To implement different page replacement algorithms
- To calculate the number of page faults
- To compare the performance of algorithms

- To analyze the efficiency of each algorithm

3. Tools and Technologies Used

- Programming Language: Python 3.x
- Code Editor: Text File
- Platform: Ubuntu (Linux)
- Libraries Used: Basic Python libraries

4. Input and Page Reference String

4.1 Description

The program takes the following inputs from the user:

- Number of memory frames
- Page reference string

These inputs simulate how pages are requested in memory.

4.2 Example Input

- Frames: 3
- Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

```
(venv) dhruvgupta@dhruvgupta:~/Downloads$ python ass3.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Reference String:
[7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
```

5. FIFO Page Replacement Algorithm

5.1 Description

FIFO replaces the page that has been in memory for the longest time. It follows a queue-based approach.

5.2 Working

- Maintain a queue of pages
- When a page fault occurs:

- Remove the oldest page
- Insert the new page

5.3 Characteristics

- Simple to implement
- Does not consider page usage frequency

```
--- FIFO ---
Page 7 -> [7]
Page 0 -> [7, 0]
Page 1 -> [7, 0, 1]
Page 2 -> [0, 1, 2]
Page 0 -> [0, 1, 2]
Page 3 -> [1, 2, 3]
Page 0 -> [2, 3, 0]
Page 4 -> [3, 0, 4]
Page 2 -> [0, 4, 2]
Page 3 -> [4, 2, 3]
Page 0 -> [2, 3, 0]
Page 3 -> [2, 3, 0]
Page 2 -> [2, 3, 0]
```

6. LRU Page Replacement Algorithm

6.1 Description

LRU replaces the page that has not been used for the longest time.

6.2 Working

- Track recently used pages
- Replace the least recently accessed page

6.3 Characteristics

- Based on temporal locality
- More efficient than FIFO

```
--- LRU ---
Page 7 -> [7]
Page 0 -> [7, 0]
Page 1 -> [7, 0, 1]
Page 2 -> [0, 1, 2]
Page 0 -> [0, 1, 2]
Page 3 -> [0, 2, 3]
Page 0 -> [0, 2, 3]
Page 4 -> [0, 3, 4]
Page 2 -> [0, 4, 2]
Page 3 -> [4, 2, 3]
Page 0 -> [2, 3, 0]
Page 3 -> [2, 3, 0]
Page 2 -> [2, 3, 0]
```

7. Optimal Page Replacement Algorithm

7.1 Description

Optimal algorithm replaces the page that will not be used for the longest time in the future.

7.2 Working

- Look ahead in the reference string
- Replace the page with farthest future use

7.3 Characteristics

- Produces minimum page faults
- Not practical in real systems (future knowledge required)

```
--- OPTIMAL ---
Page 7 -> [7]
Page 0 -> [7, 0]
Page 1 -> [7, 0, 1]
Page 2 -> [0, 1, 2]
Page 0 -> [0, 1, 2]
Page 3 -> [0, 2, 3]
Page 0 -> [0, 2, 3]
Page 4 -> [2, 3, 4]
Page 2 -> [2, 3, 4]
Page 3 -> [2, 3, 4]
Page 0 -> [2, 3, 0]
Page 3 -> [2, 3, 0]
Page 2 -> [2, 3, 0]
```

8. MRU Page Replacement Algorithm

8.1 Description

MRU replaces the most recently used page.

8.2 Working

- Track most recently accessed page
- Replace it when needed

8.3 Characteristics

- Opposite of LRU
- Useful in specific scenarios

```
--- MRU ---
Page 7 -> [7]
Page 0 -> [7, 0]
Page 1 -> [7, 0, 1]
Page 2 -> [7, 0, 2]
Page 0 -> [7, 0, 2]
Page 3 -> [7, 2, 3]
Page 0 -> [7, 2, 0]
Page 4 -> [7, 2, 4]
Page 2 -> [7, 2, 4]
Page 3 -> [7, 4, 3]
Page 0 -> [7, 4, 0]
Page 3 -> [7, 4, 3]
Page 2 -> [7, 4, 2]
```

9. Second Chance (Clock) Algorithm

9.1 Description

Second Chance improves FIFO by giving pages a second chance before replacement.

9.2 Working

- Each page has a reference bit
- If bit = 1 → give second chance
- If bit = 0 → replace page

9.3 Characteristics

- More efficient than FIFO
- Approximates LRU

```
--- SECOND CHANCE ---
Page 7 -> [7, -1, -1]
Page 0 -> [7, 0, -1]
Page 1 -> [7, 0, 1]
Page 2 -> [2, 0, 1]
Page 0 -> [2, 0, 1]
Page 3 -> [2, 0, 3]
Page 0 -> [2, 0, 3]
Page 4 -> [4, 0, 3]
Page 2 -> [4, 2, 3]
Page 3 -> [4, 2, 3]
Page 0 -> [4, 2, 0]
Page 3 -> [3, 2, 0]
Page 2 -> [3, 2, 0]
```

10. Performance Comparison

10.1 Description

The number of page faults for each algorithm is calculated and compared.

10.2 Output

The results display:

- FIFO page faults
- LRU page faults
- Optimal page faults
- MRU page faults
- Second Chance page faults

```
==================================
PAGE FAULTS COMPARISON
==================================
FIFO: 10
LRU: 9
Optimal: 7
MRU: 11
Second Chance: 9
```

11. Result Analysis

11.1 Observations

- Optimal algorithm gives the least number of page faults
- LRU performs better than FIFO in most cases
- MRU performs poorly in general cases
- Second Chance improves FIFO performance

11.2 Best and Worst Algorithm

- Best Algorithm: Optimal (minimum faults)
- Worst Algorithm: Depends on input, often MRU or FIFO

```
==================================
ANALYSIS
==================================
Best Algorithm: Optimal (Least Page Faults)
Worst Algorithm: MRU (Most Page Faults)
```

12. Challenges Faced

- Implementing correct LRU logic
- Handling future prediction in Optimal algorithm
- Managing reference bits in Second Chance
- Ensing accurate page fault calculation

13. Learning Outcomes

- Understood memory management concepts
- Learned implementation of multiple algorithms
- Analyzed algorithm efficiency
- Improved coding and debugging skills

14. Conclusion

Page replacement algorithms play a vital role in memory management. This assignment demonstrates how different algorithms behave under the same conditions. Among all, the Optimal algorithm performs best, while others provide practical trade-offs between complexity and efficiency.

Understanding these algorithms helps in designing efficient memory management systems in real-world operating systems.

15. GitHub Repository Link

👉 Github Link